

SHETH L.U.J & SIR MV COLLEGE

Aim:RULE MINING

Code:

```
# Importing the libraries
import numpy as np
import matplotlib.pyplot as plt
import pandas as pd

# Importing the dataset
dataset = pd.read_csv('iris.csv')
X = dataset.iloc[:, [0, 3]].values
y = dataset.iloc[:, -1].values

# Splitting the dataset into the Training set and Test set
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size =
0.20, random_state = 0)

# Feature Scaling
#Since our dataset containing character variables
#we have to encode it using LabelEncoder
from sklearn.preprocessing import StandardScaler
sc = StandardScaler()
X_train = sc.fit_transform(X_train)
X_test = sc.transform(X_test)

# Training the Naive Bayes model on the Training set
from sklearn.naive_bayes import GaussianNB
classifier = GaussianNB()
classifier.fit(X_train, y_train)

# Predicting the Test set results
y_pred = classifier.predict(X_test)
print("Predicted Test Results : ",y_pred)
print("~"*20)

# Making the Confusion Matrix
```

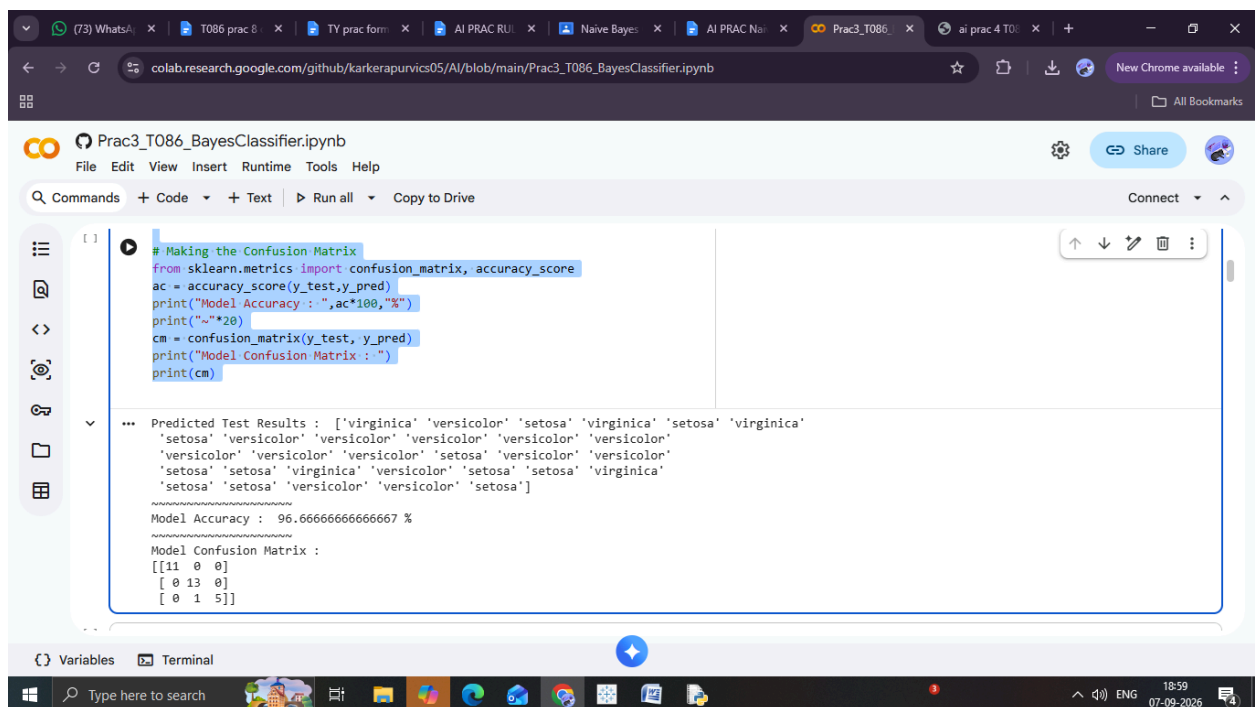
Purvi.R.Karkera

T086

SHETH L.U.J & SIR MV COLLEGE

```
from sklearn.metrics import confusion_matrix, accuracy_score
ac = accuracy_score(y_test,y_pred)
print("Model Accuracy : ",ac*100,"%")
print("~"*20)
cm = confusion_matrix(y_test, y_pred)
print("Model Confusion Matrix : ")
print(cm)
```

Output:



The screenshot shows a Google Colab notebook titled 'Prac3_T086_BayesClassifier.ipynb'. The code cell contains the same Python code as shown in the first block. The output cell displays the following results:

```
... Predicted Test Results : ['virginica' 'versicolor' 'setosa' 'virginica' 'setosa' 'virginica'
' setosa' 'versicolor' 'versicolor' 'versicolor' 'versicolor' 'versicolor'
' versicolor' 'versicolor' 'versicolor' 'setosa' 'versicolor' 'versicolor'
' setosa' 'setosa' 'virginica' 'versicolor' 'setosa' 'setosa' 'virginica'
' setosa' 'setosa' 'versicolor' 'versicolor' 'setosa']

Model Accuracy : 96.66666666666667 %

Model Confusion Matrix :
[[11  0  0]
 [ 0 13  0]
 [ 0  1  5]]
```

Code:

SHETH L.U.J & SIR MV COLLEGE

The image displays two sequential screenshots of a Google Colab notebook titled 'Prac3_T086_BayesClassifier.ipynb'. The browser address bar shows the URL: `colab.research.google.com/github/karkerapurvics05/AI/blob/main/Prac3_T086_BayesClassifier.ipynb`.

Top Screenshot: This view shows the initial code cells. The first cell imports necessary libraries: `numpy`, `pandas`, `sklearn.model_selection` (for `train_test_split`), `sklearn.naive_bayes` (for `GaussianNB`), and `sklearn` (for `metrics`). A print statement confirms the libraries are imported. The second cell loads the `iris.csv` dataset into a `pandas` DataFrame, prints a message, and displays the first five rows of the dataset.

	sepal_length	sepal_width	petal_length	petal_width	species
0	sepal_length	sepal_width	petal_length	petal_width	species
1	5.1	3.5	1.4	0.2	setosa
2	4.9	3	1.4	0.2	setosa
3	4.7	3.2	1.3	0.2	setosa

Bottom Screenshot: This view shows the next steps in the notebook. The third cell continues to display the dataset, showing rows 0 through 4. The fourth cell performs feature engineering: it drops the 'species' column from the DataFrame to create feature matrix `X`, and isolates the 'species' column as the target variable `y`. The fifth cell re-loads the `iris.csv` file and repeats the feature extraction process. The final line of code in the visible cell is `from sklearn.preprocessing import StandardScaler`, indicating the next step is to scale the features.

SHETH L.U.J & SIR MV COLLEGE

The image displays two sequential screenshots of a Google Colab notebook titled 'Prac3_T086_BayesClassifier.ipynb'. The browser address bar shows the URL: `colab.research.google.com/github/karkeraurvics05/AI/blob/main/Prac3_T086_BayesClassifier.ipynb`.

Top Screenshot: The notebook shows the initial steps of loading the 'iris.csv' file into a pandas DataFrame named 'iris_df'. The 'species' column is dropped from both 'X' and 'y'. The 'StandardScaler' from 'sklearn.preprocessing' is imported, and a 'std' object is created and fitted to the data 'X'. The output shows the 'Transformed Data' as a 150x4 array of float values.

```
[1] X = iris_df.drop("species", axis=1)
    y = iris_df["species"]

[1] iris_df = pd.read_csv("iris.csv")
    X = iris_df.drop("species", axis=1)
    y = iris_df["species"]
    from sklearn.preprocessing import StandardScaler
    std = StandardScaler()
    X = std.fit_transform(X)
    print("Transformed Data:")
    print(X)
```

Bottom Screenshot: This screenshot shows the same notebook after the 'std' object has been used to transform the data 'X'. The code cell now only contains the transformation step and the printing of the transformed data. The output is identical to the one in the top screenshot, showing the 'Transformed Data' array.

```
[1] X = std.fit_transform(X)
    print("Transformed Data:")
    print(X)
```

Both screenshots show the same output for the 'Transformed Data':

```
[[ -9.00681170e-01  1.03205722e+00 -1.34127240e+00 -1.31297673e+00]
 [ -1.14301691e+00 -1.24957601e-01 -1.34127240e+00 -1.31297673e+00]
 [ -1.38535265e+00  3.37848329e-01 -1.39813811e+00 -1.31297673e+00]
 [ -1.50652052e+00  1.06445364e-01 -1.28440670e+00 -1.31297673e+00]
 [ -1.02184904e+00  1.26346019e+00 -1.34127240e+00 -1.31297673e+00]
 [ -5.37177559e-01  1.95766909e+00 -1.17067529e+00 -1.05003079e+00]
 [ -1.50652052e+00  8.00654259e-01 -1.34127240e+00 -1.18150376e+00]
 [ -1.02184904e+00  8.00654259e-01 -1.28440670e+00 -1.31297673e+00]
 [ -1.74885626e+00 -3.56308566e-01 -1.34127240e+00 -1.31297673e+00]
 [ -1.14301691e+00  1.06445364e-01 -1.28440670e+00 -1.44444970e+00]
 [ -5.37177559e-01  1.49486315e+00 -1.28440670e+00 -1.31297673e+00]
 [ -1.26418478e+00  8.00654259e-01 -1.22754100e+00 -1.31297673e+00]
 [ -1.26418478e+00 -1.24957601e-01 -1.34127240e+00 -1.44444970e+00]
 [ -1.87002413e+00 -1.24957601e-01 -1.51186952e+00 -1.44444970e+00]
 [ -5.25060772e-02  2.18907205e+00 -1.45500381e+00 -1.31297673e+00]
 [ -1.73673948e-01  3.11468391e+00 -1.28440670e+00 -1.05003079e+00]
 [ -5.37177559e-01  1.95766909e+00 -1.39813811e+00 -1.05003079e+00]
 [ -9.00681170e-01  1.03205722e+00 -1.34127240e+00 -1.18150376e+00]
 [ -1.73673948e-01  1.72626612e+00 -1.17067529e+00 -1.18150376e+00]
 [ -9.00681170e-01  1.72626612e+00 -1.28440670e+00 -1.18150376e+00]]
```

SHETH L.U.J & SIR MV COLLEGE

The image displays two screenshots of a Google Colab notebook titled 'Prac3_T086_BayesClassifier.ipynb'. The notebook is open in a web browser, showing the 'Code' tab. The code cell contains a list of numerical data points, likely representing features for a classification task. The data is organized into rows, with each row containing multiple floating-point values. The first screenshot shows the first 20 rows of data, and the second screenshot shows the remaining rows, ending with the 40th row. The data points are arranged in a grid-like structure, with each row containing 10 values. The values are mostly in scientific notation, indicating they are small or large numbers. The notebook interface includes a menu bar with options like 'File', 'Edit', 'View', 'Insert', 'Runtime', 'Tools', and 'Help'. There is also a 'Share' button and a 'Connect' button. The bottom of the notebook shows a terminal window and a status bar with system information like temperature and time.

```
[ -3.37177559e-01 1.93700909e+00 -1.39613011e+00 -1.03003079e+00  
[ -9.00681170e-01 1.03205722e+00 -1.34127240e+00 -1.18150376e+00  
[ -1.73673948e-01 1.72626612e+00 -1.17067529e+00 -1.18150376e+00  
[ -9.00681170e-01 1.72626612e+00 -1.28440670e+00 -1.18150376e+00  
[ -5.37177559e-01 8.00654259e-01 -1.17067529e+00 -1.31297673e+00  
[ -9.00681170e-01 1.49486315e+00 -1.28440670e+00 -1.05003079e+00  
[ -1.50652052e+00 1.26346019e+00 -1.56873522e+00 -1.31297673e+00  
[ -9.00681170e-01 5.69251294e-01 -1.17067529e+00 -9.18557817e-01  
[ -1.26418478e+00 8.00654259e-01 -1.05694388e+00 -1.31297673e+00  
[ -1.02184904e+00 -1.24957601e-01 -1.22754100e+00 -1.31297673e+00  
[ -1.02184904e+00 8.00654259e-01 -1.22754100e+00 -1.05003079e+00  
[ -7.79513300e-01 1.03205722e+00 -1.28440670e+00 -1.31297673e+00  
[ -7.79513300e-01 8.00654259e-01 -1.34127240e+00 -1.31297673e+00  
[ -1.38535265e+00 3.37848329e-01 -1.22754100e+00 -1.31297673e+00  
[ -1.26418478e+00 1.06445364e-01 -1.22754100e+00 -1.31297673e+00  
[ -5.37177559e-01 8.00654259e-01 -1.28440670e+00 -1.05003079e+00  
[ -7.79513300e-01 2.42047502e+00 -1.28440670e+00 -1.44444970e+00  
[ -4.16009689e-01 2.65187798e+00 -1.34127240e+00 -1.31297673e+00  
[ -1.14301691e+00 1.06445364e-01 -1.28440670e+00 -1.44444970e+00  
[ -1.02184904e+00 3.37848329e-01 -1.45500381e+00 -1.31297673e+00  
[ -4.16009689e-01 1.03205722e+00 -1.39813811e+00 -1.31297673e+00  
[ -1.14301691e+00 1.06445364e-01 -1.28440670e+00 -1.44444970e+00  
[ -1.74885626e+00 -1.24957601e-01 -1.39813811e+00 -1.31297673e+00  
[ -9.00681170e-01 8.00654259e-01 -1.28440670e+00 -1.31297673e+00  
[ -1.02184904e+00 1.03205722e+00 -1.39813811e+00 -1.18150376e+00  
[ -1.62768839e+00 -1.74477836e+00 -1.39813811e+00 -1.18150376e+00  
[ -4.16009689e-01 2.65187798e+00 -1.34127240e+00 -1.31297673e+00  
[ -1.14301691e+00 1.06445364e-01 -1.28440670e+00 -1.44444970e+00  
[ -1.02184904e+00 3.37848329e-01 -1.45500381e+00 -1.31297673e+00  
[ -4.16009689e-01 1.03205722e+00 -1.39813811e+00 -1.31297673e+00  
[ -1.14301691e+00 1.06445364e-01 -1.28440670e+00 -1.44444970e+00  
[ -1.74885626e+00 -1.24957601e-01 -1.39813811e+00 -1.31297673e+00  
[ -9.00681170e-01 8.00654259e-01 -1.28440670e+00 -1.31297673e+00  
[ -1.02184904e+00 1.03205722e+00 -1.39813811e+00 -1.18150376e+00  
[ -1.62768839e+00 -1.74477836e+00 -1.39813811e+00 -1.18150376e+00  
[ -1.74885626e+00 3.37848329e-01 -1.39813811e+00 -1.31297673e+00  
[ -1.02184904e+00 1.03205722e+00 -1.22754100e+00 -7.87084847e-01  
[ -9.00681170e-01 1.72626612e+00 -1.05694388e+00 -1.05003079e+00  
[ -1.26418478e+00 -1.24957601e-01 -1.34127240e+00 -1.18150376e+00  
[ -9.00681170e-01 1.72626612e+00 -1.22754100e+00 -1.31297673e+00  
[ -1.50652052e+00 3.37848329e-01 -1.34127240e+00 -1.31297673e+00  
[ -6.58345429e-01 1.49486315e+00 -1.28440670e+00 -1.31297673e+00  
[ -1.02184904e+00 5.69251294e-01 -1.34127240e+00 -1.31297673e+00  
[ 1.40150837e+00 3.37848329e-01 5.35295827e-01 2.64698913e-01  
[ 6.74501145e-01 3.37848329e-01 4.21564419e-01 3.96171883e-01  
[ 1.28034050e+00 1.06445364e-01 6.49027235e-01 3.96171883e-01  
[ -4.16009689e-01 -1.74477836e+00 1.37235899e-01 1.33225943e-01  
[ 7.95669016e-01 -5.87763531e-01 4.78430123e-01 3.96171883e-01  
[ -1.73673948e-01 -5.87763531e-01 4.21564419e-01 1.33225943e-01  
[ 5.5333275e-01 5.69251294e-01 5.35295827e-01 5.27644853e-01
```

SHETH L.U.J & SIR MV COLLEGE

The image displays two sequential screenshots of a Google Colab notebook titled 'Prac3_T086_BayesClassifier.ipynb'. The browser address bar shows the URL: `colab.research.google.com/github/karkerapurvics05/AI/blob/main/Prac3_T086_BayesClassifier.ipynb`.

Top Screenshot: This view shows the initial steps of the notebook. The first code cell defines `test_size = 0.30` and `seed = 7`, then uses `train_test_split` to partition the data. A `GaussianNB()` model is instantiated and trained on the training data (`X_train, y_train`). The second code cell prints the predictions for the test set (`X_test`), resulting in a list of 15 predicted class names: `['virginica' 'versicolor' 'setosa' 'versicolor' 'versicolor' 'setosa' 'virginica' 'versicolor' 'setosa' 'versicolor' 'virginica' 'versicolor' 'setosa' 'virginica' 'setosa' 'virginica' 'virginica' 'virginica' 'setosa' 'setosa' 'versicolor' 'virginica' 'versicolor' 'versicolor' 'virginica' 'virginica' 'versicolor' 'versicolor' 'virginica' 'virginica']`.

Bottom Screenshot: This view shows the evaluation phase. The third code cell imports `accuracy_score` and `confusion_matrix` from `sklearn.metrics` and prints the confusion matrix for the predicted test results. The output is: `array([[12, 0, 0], [0, 13, 2], [0, 3, 15]])`. The fourth code cell calculates the model's score using `model.score(X_test, y_test)` and includes a link to the scikit-learn documentation for model evaluation: `#https://scikit-learn.org/stable/modules/model_evaluation.html`.

SHETH L.U.J & SIR MV COLLEGE

The image displays two screenshots of a Google Colab notebook titled 'Prac3_T086_BayesClassifier.ipynb'. The notebook is open in a web browser, showing the code editor and the output of the code.

Top Screenshot: The code cell shows the following code:

```
from sklearn.metrics import accuracy_score, confusion_matrix
metrics.confusion_matrix(predicted, y_test)

array([[12,  0,  0],
       [ 0, 13,  2],
       [ 0,  3, 15]])

#https://scikit-learn.org/stable/modules/model_evaluation.html
model_score = model.score(X_test, y_test)
model_score

#https://towardsdatascience.com/predict-vs-predict-proba-scikit-learn-bdc45daa5972
#https://scikit-learn.org/stable/search.html?q=predict_proba

y_predictProb = model.predict_proba(X_test)
print(y_predictProb)
```

The output of the code is:

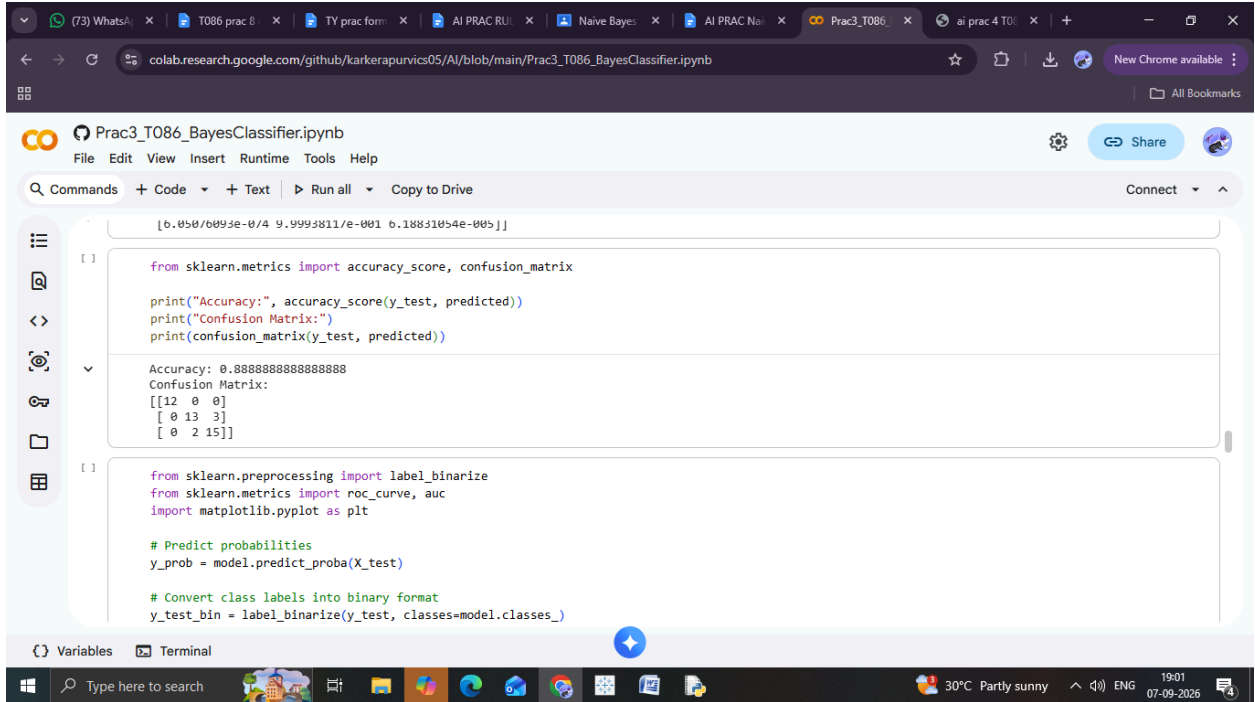
```
[[3.92493144e-144 2.36884568e-002 9.76311543e-001]
```

Bottom Screenshot: The code cell shows the same code as the top screenshot, but the output is a large array of predicted probabilities for each class:

```
[[3.92493144e-144 2.36884568e-002 9.76311543e-001]
 [8.13265358e-098 9.94457800e-001 5.54219953e-003]
 [1.00000000e+000 9.28832548e-018 2.02091585e-025]
 [2.92500338e-098 9.92623415e-001 7.37658546e-003]
 [6.84485064e-107 9.81777549e-001 1.82224506e-002]
 [1.00000000e+000 5.53559117e-015 2.47565844e-024]
 [1.69087771e-122 3.96221135e-001 6.03778865e-001]
 [4.83242183e-078 9.99939153e-001 6.08466005e-005]
 [1.00000000e+000 1.22318762e-017 3.22708548e-025]
 [3.79189139e-100 9.34697039e-001 6.53029614e-002]
 [8.95509685e-137 3.72677759e-002 9.62732224e-001]
 [1.97060262e-102 8.07038216e-001 1.92961784e-001]
 [1.00000000e+000 2.57727692e-021 1.70376019e-027]
 [1.10009591e-259 6.69852770e-015 1.00000000e+000]
 [1.00000000e+000 3.56584526e-018 1.20968818e-025]
 [2.07167354e-127 6.07779519e-002 9.39222048e-001]
 [9.55255829e-192 2.35112590e-004 9.99764887e-001]
 [5.38906944e-170 5.79858297e-004 9.99420142e-001]
 [1.00000000e+000 1.36351004e-018 1.60720713e-025]]
```

Purvi.R.Karkera
T086

SHETH L.U.J & SIR MV COLLEGE



```
[0.050/0093e-0/4 9.9993811/e-001 0.18831054e-005]]

[1]
from sklearn.metrics import accuracy_score, confusion_matrix

print("Accuracy:", accuracy_score(y_test, predicted))
print("Confusion Matrix:")
print(confusion_matrix(y_test, predicted))

Accuracy: 0.8888888888888888
Confusion Matrix:
[[12  0  0]
 [ 0 13  3]
 [ 0  2 15]]

[1]
from sklearn.preprocessing import label_binarize
from sklearn.metrics import roc_curve, auc
import matplotlib.pyplot as plt

# Predict probabilities
y_prob = model.predict_proba(X_test)

# Convert class labels into binary format
y_test_bin = label_binarize(y_test, classes=model.classes_)
```

```
from sklearn.preprocessing import label_binarize
from sklearn.metrics import roc_curve, auc
import matplotlib.pyplot as plt

# Predict probabilities
y_prob = model.predict_proba(X_test)

# Convert class labels into binary format
y_test_bin = label_binarize(y_test, classes=model.classes_)

# Plot ROC curve for each class
plt.figure(figsize=(7,6))

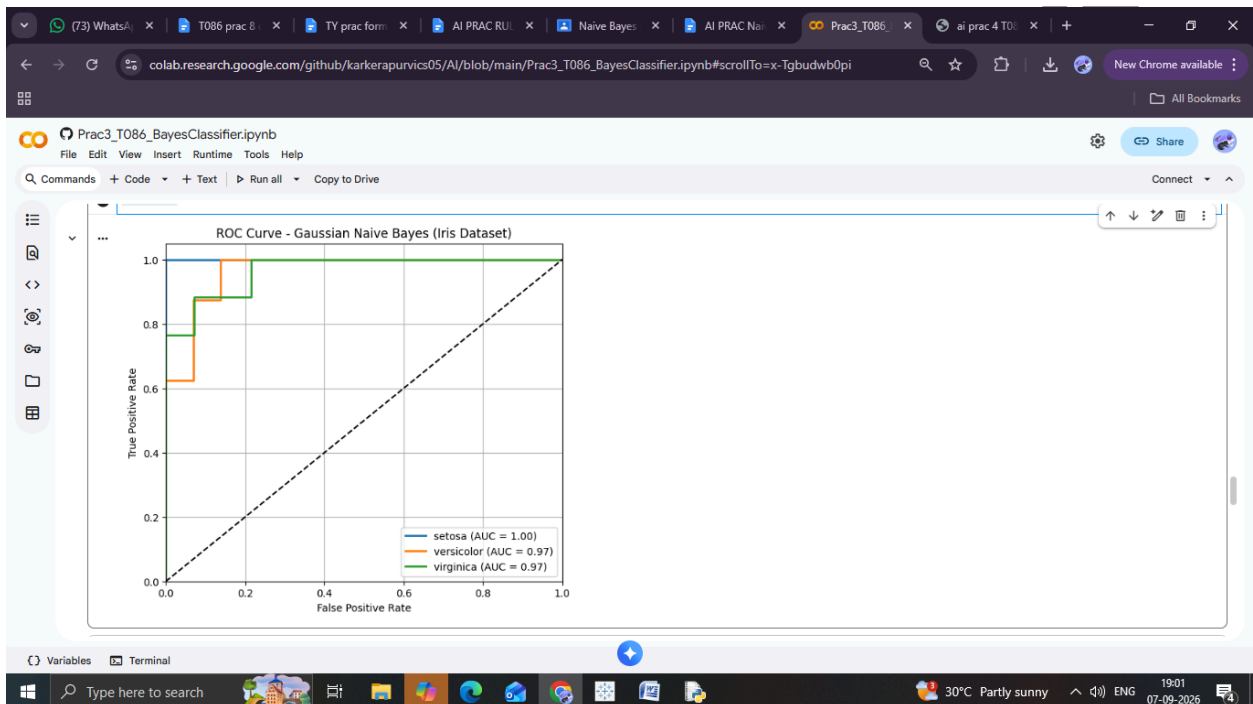
for i in range(len(model.classes_)):
    fpr, tpr, _ = roc_curve(y_test_bin[:, i], y_prob[:, i])
    roc_auc = auc(fpr, tpr)

    plt.plot(fpr, tpr,
             linewidth=2,
             label=f'{model.classes_[i]} (AUC = {roc_auc:.2f})')

# Random classifier line
plt.plot([0,1], [0,1], 'k--')
```

SHETH L.U.J & SIR MV COLLEGE

```
plt.xlim([0.0,1.0])
plt.ylim([0.0,1.05])
plt.xlabel("False Positive Rate")
plt.ylabel("True Positive Rate")
plt.title("ROC Curve - Gaussian Naive Bayes (Iris Dataset)")
plt.legend(loc="lower right")
plt.grid(True)
plt.show()
```



```
from sklearn.preprocessing import label_binarize
from sklearn.metrics import roc_curve, auc
import matplotlib.pyplot as plt

# Predict probabilities
y_prob = model.predict_proba(X_test)

# Convert multiclass labels to binary
y_test_bin = label_binarize(y_test, classes=model.classes_)

# ROC for first class (Setosa)
fpr, tpr, thresholds = roc_curve(y_test_bin[:,0], y_prob[:,0])
roc_auc = auc(fpr, tpr)
```

SHETH L.U.J & SIR MV COLLEGE

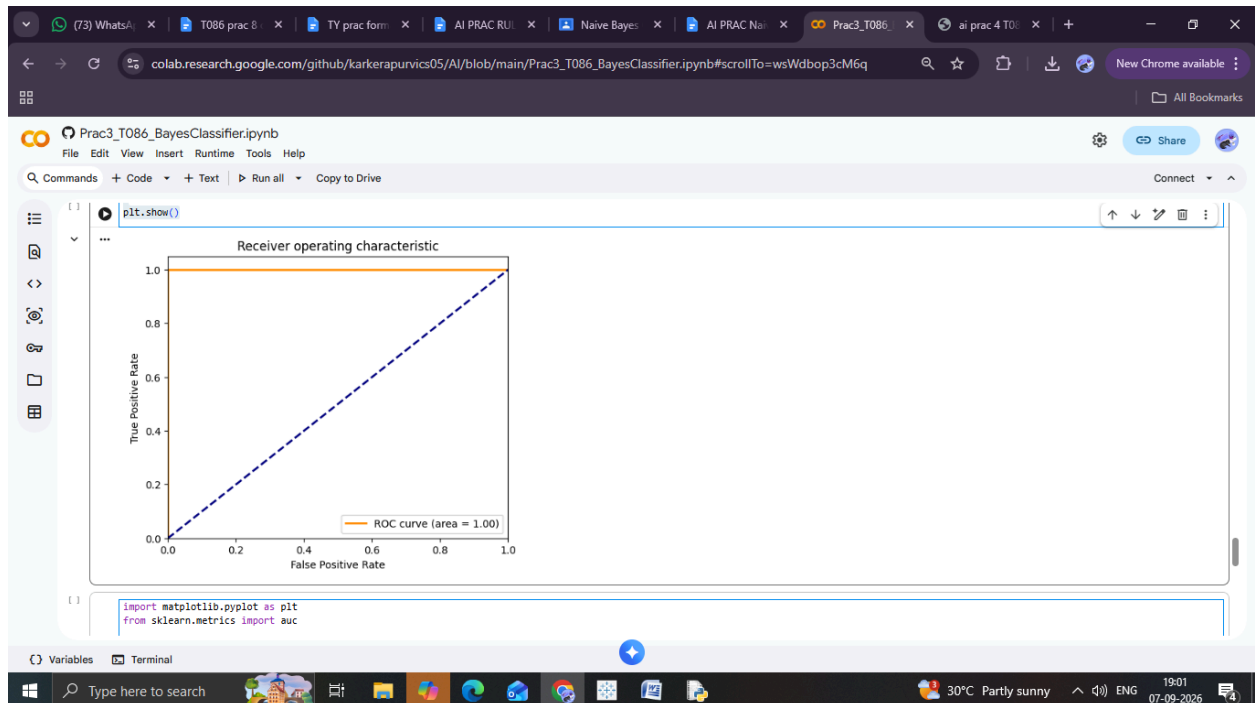
```
# Plot
plt.figure(figsize=(6,5))
plt.plot(fpr, tpr, color='darkorange',
         lw=2,
         label='ROC curve (area = %0.2f)' % roc_auc)

plt.plot([0,1],[0,1],
         color='navy',
         lw=2,
         linestyle='--')

plt.xlim([0.0,1.0])
plt.ylim([0.0,1.05])

plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.title('Receiver operating characteristic')
plt.legend(loc='lower right')

plt.show()
```



```
import matplotlib.pyplot as plt
from sklearn.metrics import auc
```

SHETH L.U.J & SIR MV COLLEGE

```
# Sample values chosen to resemble the example figure
fpr = [0.0, 0.05, 0.10, 0.15, 0.22, 0.30, 0.40, 0.52, 0.68, 1.0]
tpr = [0.0, 0.18, 0.38, 0.55, 0.68, 0.78, 0.88, 0.93, 0.98, 1.0]

roc_auc = auc(fpr, tpr)

plt.figure(figsize=(6,5))
plt.plot(fpr, tpr, color='darkorange', lw=2,
        label='ROC curve (area = %0.2f)' % roc_auc)
plt.plot([0,1], [0,1], color='navy', lw=2, linestyle='--')

plt.xlim([0.0,1.0])
plt.ylim([0.0,1.05])
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.title('Receiver operating characteristic')
plt.legend(loc='lower right')
plt.grid(False)
plt.show()
```

